Feedback on designing the proposed `std::error` type

Document #:	D2170R0
Date:	2020-05-13
Project:	Programming Language C++
Audience:	Library Evolution Group

Reply-to:	Charles Salvia <charles.a.salvia@gmail.com>

Contents

§ 1 Abstract

[P0709R4] proposes `std::error` as a mechanism for supporting deterministic static exceptions, as well as a more powerful successor to `std::error_code`. I’ve implemented the proposed `std::error`, including support for treating `std::exception_ptr` as a `TriviallyRelocatable` type that can be transported via an `std::error` object without performing an allocation. In this paper I provide feedback on the overall design of `std::error`, particularly with regard to how it might map to “legacy” error mechanisms such as dynamic exceptions and/or `std::error_code`, and also provide some discussion and rationale around design decisions and trade-offs I made while implementing `std::error` that differ from the current `status_code` based implementation proposed in [P1028R3].

This paper does not propose anything fundamentally different from [P0709R4], but merely explores certain design considerations and trade-offs that surface when implementing `std::error` and suggests what a minimal specification should probably contain based on actual implementation experience, while also discussing considerations that arise when attempting to map `std::error` to “legacy” error mechanisms such as dynamic exceptions and `std::error_code`. Finally, this paper is focused only on the design and implementation of the `std::error` type and how it maps to existing error facilities - this paper does *not* address the larger issue of deterministic static exceptions.

The implementation of `std::error` discussed here is available at:

Repos link: github.com/charles-salvia/std_error

All-in-one header-only link: github.com/charles-salvia/std_error/blob/master/all_in_one.hpp

Example usage/godbolt link: godbolt.org/z/8o1Yg6

The implementation requires at least C++14. It has been tested on GCC 4.9.2 to GCC 10, Clang 4 thru Clang 10, and MSVC 19.14 thru 19.24.

2 Design constraints

2.1 Trivial Relocation

A key feature of the proposed `std::error` type is that an instance should be no larger than two pointers. An `std::error` therefore consists of a pointer-sized type-erased object, and a constant pointer to a domain instance. This is similar to `std::error_code`, where we have an integral type code along with a pointer to an `std::error_category`, except `std::error` contains a type-erased object that could potentially represent any arbitrarily rich data. It's up to the `error_domain` object to decide how to statically cast the type-erased data to the appropriate type.

One important requirement of any type `T` that is to be type-erased and stored within an `std::error` is that `T` must be `TriviallyRelocatable`, meaning essentially that move-constructing an instance of `T` must be functionally equivalent to simply performing a bitwise copy and then replacing the moved-from instance with `T{}`. Both `std::unique_ptr` and `std::shared_ptr` could fit this requirement conceptually, as could `std::exception_ptr`.

An `std::error` object could then easily contain either an integral-type error code, or arbitrarily rich information passed in as a type-erased smart pointer which would be *move-relocated* into the pointer-sized storage contained in the `std::error` object. An `std::exception_ptr`, if implemented such that it is no larger than a raw pointer, could also be transported efficiently in an `std::error`.

However, being able to transport a type-erased smart pointer such as `std::exception_ptr` immediately brings up certain design decisions that need to be considered up-front. For example, as a type-erasing transport for arbitrary `TriviallyRelocatable` types, `std::error` must at the very least have access to a polymorphic destructor function that can invoke the destructor of the erased type. If we limit `std::error` itself to being `TriviallyRelocatable` with a deleted copy constructor, we do not need to provide any additional polymorphic functions to manage ownership of the erased type, apart from the destructor.

For example, in order to transport a `unique_ptr` in an `std::error`, we would *move-relocate* the `unique_ptr` instance into type-erased storage, and then simply bitwise-copy the type-erased storage an arbitrary number of times until the last move-constructed instance is destructed, at which point we'd need to call a polymorphic destructor that has knowledge of the erased type (implemented perhaps as a virtual function in the `error_domain`). We can set the internal `error_domain` pointer of a moved-from `std::error` instance to `nullptr` to disable destructors of moved-from error objects. This is how the `status_code` based implementation of `std::error` proposed in [P1028R3] is implemented. This design also requires that `std::error` itself is move-only.

2.2 Is std::error move-only?

Should `std::error` be required to be move-only? The original proposal [P0709R4] doesn't explicitly say that `std::error` is move-only; rather, it only specifies that `std::error` should be trivially relocatable. However, this doesn't necessarily mean `std::error` must be move-only. A generic copy constructor for some erased type `T` could be safely implemented by copy-constructing a temporary, and then move-relocating it into the destination storage. Regardless, the current `status_code` based implementation of `std::error` proposed in [P1028R3] is indeed move-only, but provides a `clone()` function for explicitly copying an error instance.

The implementation of `std::error` submitted here is both trivially-relocatable and copy-constructible. (Note that many trivially relocatable types are copy-constructible, but not *trivially* copy-constructible, such as

`std::shared_ptr`.) Implementing a copy-constructible version of `std::error` requires that we call the copy-constructor of the type-erased object each time the `std::error` object itself is copied. This can be efficiently implemented by providing the `error_domain` instance with three function pointers - a copy constructor, move constructor, and destructor. (Again, the copy construction would need to be implemented in terms of a copy, followed by a move-relocate.) Error domains that use trivially-copyable types, such as enums or integral types, can set the function pointers to `nullptr`. Since error domains are expected to be `constexpr`, the call to the copy/move-constructor or destructor can be optimized out, and since statically thrown errors are essentially return-by-value, copying would be elided very often anyway.

Regardless, it remains an open question whether `std::error` should be copy-constructible. As a general replacement for both `std::error_code` as well as dynamic exceptions, it seems we should default to allowing `std::error` to support copy construction unless there is a very compelling reason not to. Both `std::error_code` and `std::exception_ptr` are copy-constructible, and there seems no reason to limit `std::error` to being move-only, apart from the requirement that any type-erased object used with `std::error` would need to be trivially-relocatable *and* copy-constructible. This rules out `std::unique_ptr`, but allows `std::exception_ptr`.

As a general mechanism for conveniently transporting arbitrary data via a copy-constructible `std::error`, it would be nice to be able to easily store a type-erased reference counted smart pointer. Unfortunately, we cannot directly store an `std::shared_ptr` in the single-pointer-sized storage provided by `std::error`, since a `std::shared_ptr` implementation typically requires at least 2 pointers due to aliasing support [N2351]. Rather, we would need a simpler reference counting smart pointer type, perhaps using intrusive reference counting, so that it could fit into single-pointer-sized storage. (The implementation presented here includes a reference-counting intrusive smart pointer to demonstrate transporting arbitrary data via an `std::error`). Note that we don't have the same problem with `std::exception_ptr`, which can be implemented as a move-relocatable type with single-pointer size, and in fact is single-pointer-size in mainstream implementations such as [GNU libstdc++].

3 Mapping "Legacy" Error Mechanisms to std::error

3.1 Mapping std::error_code to std::error

The original proposal [P0709R4] discusses mapping standard dynamic exception types to `std::error`, and also implies that `std::errc` enums will be inherited from `<system_error>` to represent a "generic" domain of errors. However, it doesn't discuss mapping an arbitrary `std::error_code` to an `std::error` object. Presumably, in the interest of easing adoption of `std::error`, we would want any possible `std::error_code` to be convertible to an `std::error`.

We can efficiently convert `std::error_code` objects that use standard error categories to an `std::error` by simply storing `ErrorCodeEnum`s in `std::error` storage and providing matching `error_domain` objects for each standard `error_category`. Perhaps we would have a `generic_domain` that represents error values using `std::errc`, exactly like `std::generic_category()` does now.

However, we cannot efficiently represent an arbitrary `std::error_code` object that may potentially have any arbitrary user-defined `error_category`. We need at least 2 pointers worth of storage to represent an arbitrary `std::error_code` - one for the code itself and the other for a pointer to the `error_category`. Since we only have one pointer worth of storage in an `std::error`, we'd need a heap allocation to map an arbitrary `std::error_code` to an `std::error`. The implementation submitted here provides matching error domains for standard categories (such as `std::generic_category`), and uses heap allocation managed by a reference counted pointer to store `std::error_code`s with custom categories in an `std::error` object.

Finally, an `std::error_code` may potentially store a zero-value `ErrorCodeEnum` to represent "not an error", whereas an `std::error` *always* represents an error. If we want to provide mappings between `std::error_code` and `std::error` we essentially have two choices to handle "not an error" error codes. We can either consider it Undefined Behavior (in the same way that throwing a null `std::exception_ptr` is undefined behavior), or more preferably, we map it to a special error code/domain that represents an invalid error, such as `errc::bad_error`.

§ 3.2 Mapping dynamic exceptions to std::error

The original proposal [P0709R4] discusses mapping standard dynamic exception types to `std::error`. Note that when we talk about mapping a dynamic exception to an `std::error`, we're mostly talking about doing so in the context of a weak-equality (semantic-equivalence) comparison. An `std::error` can *directly* store an `std::exception_ptr`, so there's no need to perform any "mapping" at all for the purposes of simply converting a dynamic exception to an `std::error`. However, we probably want to be able to use the `==` operator to perform a semantic comparison with an `std::error` against a set of generic enums such as `std::errc`.

In this context, it would be useful to map, for example, `std::bad_alloc` to `std::errc::not_enough_memory`, so that a dynamic exception converted to an `std::error` can be tested for weak-equality or semantic equivalence. The original proposal mentions the need to standardize these mappings.

It turns out that providing mappings from the set of standard C++ dynamic exceptions to the set of `std::errc` enums is not easy to do in a way that consistently results in meaningful error translations. At first glance it may seem promising: we can map `std::invalid_argument` to `std::errc::invalid_argument` of course, `std::bad_alloc` to `std::errc::not_enough_memory`, and `std::domain_error` to `std::errc::argument_out_of_domain`, to name a few obvious mappings.

The documentation for the `status_code` based implementation of `std::error` suggests the following mappings:

```
std::invalid_argument     →    errc::invalid_argument
std::domain_error         →    errc::argument_out_of_domain
std::length_error         →    errc::argument_list_too_long
std::out_of_range         →    errc::result_out_of_range
std::logic_error          →    errc::invalid_argument
std::overflow_error       →    errc::value_too_large
std::range_error          →    errc::result_out_of_range
std::runtime_error        →    errc::resource_unavailable_try_again
std::bad_alloc            →    errc::not_enough_memory
```

The implementation submitted here suggests a slightly modified mapping. However, both mappings are incomplete, reductive, and contain some highly questionable translations such as `std::runtime_error` → `errc::resource_unavailable_try_again`. Additionally, this only provides mappings for standard exceptions included under `<stdexcept>`. But how would we map something like `std::regex_error`, `std::bad_cast` or `std::bad_variant_access`? In the proposed mapping above, an `std::regex_error`, which inherits from `std::runtime_error`, would be mapped to `errc::resource_unavailable_try_again`, which conveys no useful semantic information about the original error and is probably misleading or confusing to anyone familiar with the POSIX error codes `EAGAIN` or `EWOULDBLOCK`.

§ 3.3 POSIX != generic

The root of the problem here is that we are probably wrongly assuming that the `std::errc` enums really constitute a set of truly "generic" error codes. This faulty assumption is understandable since, after all, these error codes are lifted from POSIX and are designed to provide portability, which is why `std::errc` codes are used with `std::generic_category` in `<system_error>`. But the reality is the POSIX error codes that the `std::errc` enums map to were devised with the goal of defining a set of generic error codes that might usefully map to the sort of errors that an *Operating System API* might produce. (And not just any OS API really, but specifically a UNIX-flavored OS API.) As a set of generic *system* error codes, POSIX seems to have provided a useful set of mostly portable codes for `<system_error>` to adopt. But again these codes were lifted from *Operating System API* errors - *not* generic application errors. These codes include error conditions for circumstances that are completely beyond the scope of the current C++ standard, such as `std::errc::broken_pipe` and `std::errc::inappropriate_io_control_operation`.

Of course, the intention of `<system_error>` was never to provide a set of truly *generic* error codes - it was to provide a set of generic *system* error codes that could be portably communicated in standard C++. In that capacity, using the POSIX error codes for `std::errc` was a reasonable choice. But regardless, `<system_error>` is not meant to provide a set of "generic" application-level error conditions, and so this family of error codes is probably not suitable for mapping *application-level* errors, the very type of errors that standard C++ dynamic exceptions represent. A hypothetical set of error codes that represent truly generic *application* errors would look more like what we get in `<stdexcept>`: maybe something like `generic_errc::runtime_error`, `generic_errc::out_of_range`, `generic_errc::size_error`, etc. This is the same sort of "generic error" taxonomy found in typical standard exception libraries across many languages, such as Python's builtin `RuntimeError`, `OverflowError`, `IndexError`, etc. A set of generic error codes mapping to C++ exceptions should probably look more like this and less like something that includes `inappropriate_io_control_operation` and `bad_file_descriptor`.

In fact, Boost.Python appears to have better luck mapping standard C++ exceptions to Python exceptions, since the error domain (generic application runtime errors) is essentially the same. But `std::errc` represents a *different* error domain - the error domain of a UNIX-derived OS API, with pipes, file descriptors and `ioctl` calls. To the extent that the concerns of an Operating System API overlap with the concerns of a user-level application (mostly within the domain of argument validation) we can find meaningful mappings from `<system_error>` to standard C++ exceptions. But beyond that it becomes a stretch, with very little semantic information conveyed by the error code.

While POSIX.1 attaches certain broad semantics to each error macro, the majority of these codes were clearly designed around common failures that may occur within a networking stack or file system, or through exhaustion of OS resources or incorrect user space usage of an OS API or service. Apart from the C++ Networking TS or Filesystem library, only a few codes out of the 80 or so codes defined in POSIX.1 are applicable to the same conceptual error domain as most standard C++ exceptions. These probably include `EINVAL`, `ERANGE`, `EDOM`, `E2BIG`, and `ENOMEM`. These alone are clearly insufficient to create meaningful mappings across all standard C++ exceptions. Note there is not even a POSIX code that clearly maps to "index out of bounds": we'd have to settle for `EINVAL` which is reductive, or else abuse the intended meaning of `ERANGE`.

Since `std::error` is clearly meant as "one error type to rule them all" - as the eventual successor of both `std::error_code` and dynamic exceptions in general, it seems we clearly need something a lot broader than `<system_error>` to use as a family of "generic" error codes. The current `status_code` based implementation of `std::error` proposed in [P1028R3] reuses the `std::errc` codes (with one slight modification), and expects all error domains to be able to map errors to one of these "generic" codes. But for many error domains there simply isn't anything useful in POSIX to map to. How again do we map an `std::regex_error` to a POSIX code? Do we just use `errc::invalid_argument` or something and give up on providing any meaningful mapping?

I would suggest two things: firstly, to review the idea that every possible `std::error` domain is to be expected to map to some "universal" set of generic error codes in the first place. I argue that this requirement is likely to be impractical for many error domains, and if enforced would simply lead to many useless error mappings that convey next to zero actual information about the semantics of the original error.

Secondly, if we do want to provide something like a set of "universal" generic error codes that any error domain could feasibly map to, *and* if we also want to have a set of error codes available that enables useful weak-equality comparisons with an `std::error` that stores an `std::exception_ptr`, we would probably want to define a new, more universal set of error code enums that are not specifically designed for Operating System API errors. This set of error codes should more closely map to the set of exceptions defined in `<stdexcept>` and other standard headers, and include codes for C++-specific error conditions such as an invalid `dynamic_cast`.

The approach submitted here (and used in the submitted implementation) is to begin by defining a set of error code enums specifically meant to represent all standard dynamic exceptions. Note that many standard exceptions are de-facto equivalent to an error code enum, in the sense that they convey no information other then their *type* and a static message. For example, `std::bad_variant_access` or `std::bad_cast` (or any other exception type that provides only a default constructor, copy constructor, and static `what()` message), convey no information beyond their type and the static message associated with their type. Thus they are representable as semantically equivalent

error codes with zero loss of information. Other exception types (such as `std::runtime_error`) take a per-instance message string, and so cannot be represented as an error code without information loss. However, since `std::error` can store an `std::exception_ptr` which can convey the per-instance message, there is no problem here. Regardless, mapping all standard exceptions to a special set of dynamic exception error codes may be useful in bridging dynamic exceptions and static errors, especially with regard to weak-equality testing. These dynamic exception error code enums may look something like the following:

```
enum class dynamic_exception_errc
{
    runtime_error,
    domain_error,
    invalid_argument,
    length_error,
    out_of_range,
    range_error,
    overflow_error,
    underflow_error,
    system_error,
    bad_alloc,
    bad_array_new_length,
    bad_optional_access,
    bad_typeid,
    bad_any_cast,
    regex_error,
    bad_cast,
    bad_weak_ptr,
    bad_exception,
    bad_variant_access,
    unspecified_exception
};
```

This would specifically map dynamic exceptions to a set of error codes that could be usefully employed to test for weak equality. On top of this, I would also suggest providing a set of truly "generic" application level error code enums, perhaps using the enum class `generic_errc`. Ideally, `std::errc` would have been named `std::posix_errc` - however, again `<system_error>` was designed foremost within the context of the Operating System error domain, and was probably not initially conceived of with generic application level error codes in mind.

Defining a set of generic application level error codes is beyond the scope of this paper, and the submitted implementation does not even attempt this. However, a hypothetical set of generic error codes would probably look something like the above list of dynamic_exception error codes. As such, a better approach may simply be to get rid of the idea of a separate `dynamic_exception_errc` set of codes and just make these the generic codes. Domains that have more specific error-reporting requirements, such as `std::regex_error` could also implement separate `error_domain`s.

§ 4 The submitted std::error implementation

The submitted `std::error` implementation is not meant to represent a formal proposal, but to serve as a hopefully useful contribution and alternative reference implementation of `std::error` that can assist with design discussions going forward. It can also serve as an initial sketch of a specification for `std::error` and associated types. The submitted implementation supports all the requirements laid out in [P0709R4], namely:

1. `std::error` always represents a failure
2. `sizeof(std::error) == sizeof(std::intptr_t) * 2`
3. It uses `constexpr` error_domain discriminants to implement polymorphic behavior
4. It is able to represent all causes of failure within the C++ standard library

5. It is type-erased, allocation-free, and trivially-relocatable
6. It can be used in a header-only library (unlike `std::error_category`)
7. It has a `==` operator that provides weak-equality comparison (semantic-equivalence)

The submitted implementation includes an opt-in `is_trivially_relocatable` trait that enables `std::exception_ptr` to be stored directly within an `std::error` without an additional allocation (provided the implementation of `std::exception_ptr` is only one pointer in size, such as the [GNU libstdc++] implementation shipped with GCC).

§ 4.1 Differences with the status_code library implementation proposed in [P1028]

The submitted implementation differs with the current `status_code` based implementation proposed in [P1028R3] as follows:

1. This implementation is a stand-alone implementation of `std::error` - it does not implement any larger status code or error code library
2. This implementation makes `std::error` copy-constructible instead of move-only
3. This implementation does not require all error domains to be able to map errors to a universal "generic" error code
4. For the purpose of providing error code enums used for weak-equality comparisons with `std::error` objects transporting a thrown dynamic exception, this implementation defines a new `dynamic_exception_errc` enum class. (It also supports semantic comparison with `std::errc` such that, e.g. a thrown `std::invalid_argument` will compare semantically equivalent to an `std::errc::invalid_argument`, but for the reasons discussed above I consider `std:errc` to be inadequate for serving as a set of universal generic error codes.)
5. Error domain ID's are 128-bit integers instead of 64-bit. With 64-bit, the chance of a collision approaches 50% when you reach about ~4 billion randomly generated error domain IDs. While it's obviously absurdly unlikely any single application will ever have that many error domains, consider a future where distributed applications transmit serialized `std::error` objects over the wire. In this case domain IDs should be globally unique, so 128-bit is probably more appropriate.

With regard to the weak-equality comparison algorithm, this implementation submits a simpler algorithm than the algorithm implemented in [P1028R3]. The algorithm implemented in [P1028R3] is as follows:

```
Given: error E1 with error_domain D1, and error E2 with error_domain D2

If both D1 and D2 are non-null:
   If D1.WeakEqualityCompare(E1, E2): return true
   Else If D2.WeakEqualityCompare(E2, E1): return true
   Else:
      Convert E2 to GenericCode G2
      If D1.WeakEqualityCompare(E1, G2): return true

      Convert E1 to GenericCode G1
      If D2.WeakEqualityCompare(E2, G1): return true

return true if both D1 and D2 are null
```

Note this algorithm requires error domains to implement conversion from some arbitrary error type to a universal set of "generic" errors (specified in [P1028R3] as identical to the POSIX error codes defined in `<system_error>`). The implementation submitted here does not require that all error types must be representable by a single set of generic codes. I have argued that this is very likely impractical for certain error domains, and if enforced would

merely lead to many useless mappings to “generic” codes that convey close to zero information about the semantics of the original error.

Additionally, the implementation submitted here does not permit an error object to have a `nullptr` for an error_domain. In [P1028R3], `error` objects are move-only and therefore a “moved-from” `error` has a null domain pointer. However, `error` objects in this implementation are copy-constructible; errors that transport non-trivially-copyable or non-trivially-movable types will provide polymorphic move and copy constructors. It is up to the domain-specific move constructor to decide how to represent a “moved-from” type-erased error value internally. Therefore, there is no reason to allow an error object to contain a null domain pointer.

With null domain pointers disallowed, and without the requirement that all error domains provide a mapping to a set of universal generic error codes, the weak-equality comparison algorithm implemented here is simply:

```
Given: error E1 with error_domain D1, and error E2 with error_domain D2

If D1.WeakEqualityCompare(E1, E2): return true
Else If D2.WeakEqualityCompare(E2, E1): return true
return false
```

§ 4.2 A minimal sketch of an std::error specification

The most significant types specified here are the `error` type itself, along with the `error_domain` abstract base type:

```
class error_domain
{
public:
   constexpr error_domain_id id() const noexcept;

   virtual string_ref name() const noexcept = 0;
   virtual bool equivalent(const error& lhs, const error& rhs) const noexcept = 0;
   virtual string_ref message(const error&) const noexcept = 0;
   virtual void throw_exception(const error& e) const;

protected:
   constexpr explicit error_domain(error_domain_id id) noexcept;

   constexpr error_domain(error_domain_id id, error_resource_management erm) noexcept;

   error_domain(const error_domain&) = default;
   error_domain(error_domain&&) = default;
   error_domain& operator = (const error_domain&) = default;
   error_domain& operator = (error_domain&&) = default;
   ~error_domain() = default;
};

constexpr bool operator == (const error_domain& lhs, const error_domain& rhs) noexcept;
constexpr bool operator != (const error_domain& lhs, const error_domain& rhs) noexcept;

// Exposition only
template <class T>
concept TypeErasable = is_trivially_relocatable_v<T> && (sizeof(T) <= sizeof(std::intptr_t));

class error
{
public:
   constexpr error() noexcept;
   constexpr error(const error& e);
   constexpr error(error&& e);

   template <class T> requires TypeErasable<T>
   constexpr error(const error_value<T>& v, const error_domain& d) noexcept(
```

```
      std::is_nothrow_copy_constructible_v<T> && std::is_nothrow_move_constructible_v<T>
   );

   template <class T> requires TypeErasable<T>
   constexpr error(error_value<T>&& v, const error_domain& d) noexcept(
      std::is_nothrow_move_constructible_v<T>
   );

   constexpr error(error_value<> v, const error_domain& d) noexcept;

   template <class A, class... Args>
      requires /* make_error((A&&)a, (Args&&)args...) via ADL returns an error object */
   constexpr error(A&& a, Args&&... args) noexcept(/* unspecified */);

   error& operator = (const error& e);
   error& operator = (error&& e) noexcept;
   ~error();

   const error_domain& domain() const noexcept;

   string_ref message() const noexcept;

   [[noreturn]] void throw_exception() const;
};

bool operator == (const error& lhs, const error& rhs) noexcept;
bool operator != (const error& lhs, const error& rhs) noexcept;
```

The class template `error_value` serves as a mechanism for constructing an `error` object from a TypeErasable object and an error_domain. In practice this class would likely only be used by `error_domain` implementors.

```
// Transports a type-erasable object of type T
template <class T = void>
class error_value
{
public:
   using value_type = T;

   constexpr error_value(const T& v) noexcept(
      std::is_nothrow_copy_constructible_v<T>
   );

   constexpr error_value(T&& v) noexcept(
      std::is_nothrow_move_constructible_v<T>
   );

   constexpr const T& value() const & noexcept;
   constexpr T& value() & noexcept;
   constexpr const T&& value() const && noexcept;
   constexpr T&& value() && noexcept;
};

// Type-erasing specialization: type erases and transports a type-erased object
template <>
class error_value<void>
{
public:
   template <class T> requires TypeErasable<std::remove_cvref_t<T>>
   constexpr error_value(T&& v) noexcept(/* unspecified */);

   template <class T> requires TypeErasable<T>
   constexpr error_value(const error_value<T>& v) noexcept(/* unspecified */);

   template <class T> requires TypeErasable<T>
```

```
   constexpr error_value(error_value<T>&& v) noexcept(/* unspecified */)
};
```

Additionally, an arbitrary type can be made implicitly convertible to an `error` via the `make_error` ADL-based customization point function.

The following free functions are provided for extracting the type-erased object from an `std::error` instance. This function is efficient but unsafe, as no type-checking is performed, so it should generally only be used directly by an `error_domain` object that has information about the erased type. These functions only participate in overload resolution if `T` is `TypeErasable`.

```
// Returns an instance of T that was directly copy-constructed from the internal error storage
template <class T>
constexpr T error_cast(const error& e) noexcept(/* unspecified */);

// Returns an instance of T that was directly move-constructed from the internal error storage
template <class T>
constexpr T error_cast(error&& e) noexcept(/* unspecified */);
```

Finally, an `error_resource_management` class template is used to provide an `error_domain` with a copy-constructor, move-constructor, and destructor for a type-erased object.

```
struct error_resource_management
{
   using copy_constructor = error_value<void>(*)(const error&);
   using move_constructor = error_value<void>(*)(error&&);
   using destructor = void(*)(error&);

   constexpr error_resource_management() noexcept;
   constexpr error_resource_management(copy_constructor, move_constructor, destructor) noexcept;
};
```

For convenience, a `default error_resource_management` variable template is provided which automatically generates copy/move constructors and a destructor for an arbitrary `TypeErasable` type `T`:

```
template <class T>
inline constexpr default_error_resource_management_t<T> default_error_resource_management {};
```

The `default_error_resource_management_t` class must behave *as if* it was implemented as follows:

```
namespace detail {

   struct default_error_constructors
   {
      template <class T>
      static error_value<> copy_constructor(const error& e)
      {
         T value = error_cast<T>(e);
         return error_value<>{std::move(value)};
      }

      template <class T>
      static error_value<> move_constructor(error&& e)
      {
         return error_value<>{error_cast<T>(std::move(e))};
      }

      template <class T>
      static void destructor(error& e) noexcept
      {
```

```
         unspecified_erased_error_ref value = /* unspecified: get reference to internal storage */
         std::launder(reinterpret_cast<T*>(&value.storage))->~T();
      }

      template <class T>
      constexpr static error_resource_management::copy_constructor copy() noexcept
      {
         return std::is_trivially_copy_constructible_v<T> ?
            nullptr : &copy_constructor<T>;
      }

      template <class T>
      constexpr static error_resource_management::move_constructor move() noexcept
      {
         return std::is_trivially_move_constructible_v<T> ?
            nullptr : &move_constructor<T>;
      }

      template <class T>
      constexpr static error_resource_management::destructor destroy() noexcept
      {
         return std::is_trivially_destructible_v<T> ?
            nullptr : &destructor<T>;
      }
   };
}

template <class T>
struct default_error_resource_management_t : error_resource_management
{
   constexpr default_error_resource_management_t() noexcept
      :
      error_resource_management{
         detail::default_error_constructors::copy<T>(),
         detail::default_error_constructors::move<T>(),
         detail::default_error_constructors::destroy<T>()
      }
   {}
};
```

As with the `status_code`-based implementation proposed in [P1028R3], a `string_ref` type along with a reference-counted shared-ownership version (here called `shared_string_ref`) is provided for storing and transporting error messages.

The submitted implementation defines the following `error_domain`s:

1. **generic_domain**: transports an `std::errc`, supports weak-equality comparison with `std::errc`, `dynamic_exception_errc`, `std::error_code` or an `error` object transporting any of these or an `error` object transporting an `std::exception_ptr`.

2. **error_code_domain**: transports an `std::error_code`, supports weak-equality comparison to `std::error_code`, `std::errc`, or an `error` object transporting any of these or an error object transporting an `std::exception_ptr`

3. **dynamic_exception_domain**: stores an `std::exception_ptr`, supports weak-equality comparison with `dynamic_exception_errc`, `std::errc`, `std::error_code`, or an `error` object transporting any of these or an `error` object transporting an `std::exception_ptr`.

4. **dynamic_exception_code_domain**: stores a `dynamic_exception_errc`, supports weak-equality comparison with `dynamic_exception_errc`, `std::errc`, `std::error_code` or an `error` object transporting any of these or an error object transporting an `std::exception_ptr`.

Per the above discussion of POSIX error codes as insuitable for a generic error domain, I would alternatively propose that `generic_domain` be changed to `posix_domain`, and `dynamic_exception_code_domain` be reworked into an error_domain for generic application-level error codes, including individual codes for each standard C++ exception type. Additionally, I would propose including `system_domain` which, similarly to the current `std::system_category`, transports error codes returned from an Operating System API and provides semantic comparison to the portable set of POSIX errors defined in `std::errc`.

5 References

[GNU libstdc++] GNU libstdc++ implements a single-pointer-sized exception_ptr.
https://godbolt.org/z/wBVejv

[N2351] Peter Dimov and Beman Dawes. 2007. Improving shared_ptr for C++0x, Revision 2.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2007/n2351.htm#aliasing

[P0709R4] Herb Sutter. 2019. Zero-overhead deterministic exceptions.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p0709r4.pdf

[P1028R3] Niall Douglas. 2020. SG14 status_code and standard error object.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1028r3.pdf